Operational Architecture and Governance of the Stela Platform: A Deep Dive into DP-OPS-0023 and Phase 3 Integration

I. Executive Summary

The Strategic Transformation of Evolutionary Legacy Architecture (STELA) represents a sophisticated, governance-first approach to modernizing legacy software systems.[1] Unlike conventional refactoring methodologies that prioritize framework migration or code translation, the Stela paradigm centers on **provenance**, **explainability**, and **strict jurisdictional control** between human operators and AI agents.[3] This report provides an exhaustive analysis of the platform's current state (Phase 2), the architectural pivots necessitated by the transition to Phase 3, and the specific execution plan for **DP-OPS-0023**, a critical bridge task designed to align the operational binaries with the new "Mechanistic Agent Model".[3]

The analysis is grounded in a deep forensic review of the repository's constitutional documents (TRUTH.md, AGENTS.md), its historical ledger (SoP.md), and its operational shell scripts (ops/bin/*, ops/lib/*). The findings indicate that the platform is currently in a state of "Planned Equilibrium" following the execution of DP-OPS-0022, which consolidated the library and restored the system gate (ops/bin/open) to a static state.[3] However, this restoration has created a temporary functional gap: the project-level workflow binary (ops/bin/project) retains legacy calls to features that have been removed from the system gate, creating a "Dead Pointer" risk that DP-OPS-0023 must resolve.[3]

This report articulates the strategy for "Project-Elected Mechanisms," where agent personae are no longer injected by the system root but are instead elected by individual project specifications (STELA.md). This shift from "System Injection" to "Project Election" is the defining characteristic of Phase 3, enabling the platform to scale indefinitely without polluting the global context.[3] The comprehensive Dispatch Packet (DP) drafted herein provides the exact technical blueprint for this transition, adhering strictly to the TASK.md template and the "Anti-Drift" governance standards mandated by the Stela Constitution.[3]

II. The Constitutional Architecture (The Truth Layer)

The resilience of the Stela platform derives from its "Truth Layer"—a set of immutable, text-based invariants that function as the constitution for all autonomous agents. In a

stateless operational environment where AI workers have no persistent memory, these documents provide the "Hard Constraints" necessary to prevent hallucination and architectural drift.[3]

2.1. The Supreme Court: TRUTH.md

TRUTH.md is the "Single Source of Truth" (SSOT) regarding the physical structure and filing doctrine of the repository. It operates on the axiom of **Precedence**: if a user instruction conflicts with TRUTH.md, the agent is contractually obligated to stop and request clarification.[3]

The Filing Doctrine: The repository enforces a rigid separation of concerns at the root directory level. This is not merely an organizational preference but a safety mechanism to prevent "Context Pollution".[3]

Directory	Purpose	Agent Access Permissions
ops/	**Run**: Binaries, manifests, automation logic.	**Read/Execute** (Write protected without DP).
docs/	**Explain**: Manuals, maps, and rationale.	**Read/Write** (Requires SoP entry for changes).
projects/	**Work**: The payload code (CMS internals).	**Read/Write** (Scoped strictly by DP).
storage/	**Trash**: Ephemeral, local-only artifacts.	**Read/Write** (Excluded from git tracking).

This doctrine ensures that "Runtime" (ops/) never bleeds into "Explanation" (docs/), and "Work" (projects/) is sandboxed from the "System".[3]

2.2. The Jurisdictional Framework: AGENTS.md

If TRUTH.md is the law of the land, AGENTS.md is the staffing protocol. It defines the "Relationship between the authority of the human and the logic of the AI".[3] The framework distinguishes three distinct roles to manage risk and accountability.[4]

1. **Operator (Human):** The Operator is the sole entity with "Git Authority" to perform commits, pushes, and merges. They hold the "Final Decision" on all scope and approval. This role is designed to be the "Human-in-the-Loop" safeguard against autonomous

drift.[3]

2. **Integrator (Lead AI):** The Integrator acts as the "Guardian of Governance." Its primary duties are auditing, drift detection, and the generation of Dispatch Packets (DPs). The Integrator does not execute code changes; it defines the *intent* of the changes.[3]
3. **Contractor (Guest AI):** The Contractor is the "Execution Arm." It is strictly limited to drafting logic and implementation details within a scope defined by the Integrator. The Contractor is forbidden from inventing new patterns and must "Reuse-First" by cross-referencing existing templates in ops/.[3]

Behavioral Logic Standards:

AGENTS.md imposes a "Linguistic Precision" standard on all AI agents.

- **Contraction Prohibition:** Agents must not use contractions (e.g., "don't," "can't"). This forces a formal, rhythmic cadence that is easier to audit programmatically and less prone to ambiguity.[3]
- **Quantitative Reporting:** Deviations must be reported numerically (e.g., "execution path deviated by 14.7%") rather than subjectively.[3]
- **Absolute Literalism:** Instructions are to be interpreted with 100% fidelity. "Vibes" are explicitly forbidden; only concrete tasks are recognized.[3]

2.3. The Historical Ledger: SoP.md

The "State of Play" (SoP.md) is the repository's merge-grade readiness ledger. It tracks the "Why" behind every change to the Canon.

- **The SoP Rule:** A change to any Canon file (TRUTH.md, TASK.md, AGENTS.md) is considered a "Governance Event." Such events are invalid unless accompanied by a corresponding entry in SoP.md within the same Pull Request.[3]
- **Drift Mitigation:** By forcing agents to log the *intent* of a governance change, the system prevents "Creeping Normality" where small, undocumented changes slowly erode the system's integrity.[4]

III. The Operational Subsystem (The Ops Layer)

The ops/ directory functions as the "Operating System" for the repository. It contains the binaries (ops/bin/) and libraries (ops/lib/) that enforce the constitution programmatically.[3]

3.1. The System Gate: ops/bin/open

ops/bin/open is the "Session Start" command. It generates the "Open Prompt"—a massive, context-rich instruction block that rehydrates the AI agent.[3]

- **Freshness Gate:** The script performs a git status check (git rev-parse HEAD, git status --porcelain) to ensure the agent is working on the correct base. If the tree is dirty, the

session start is flagged as risky.[3]

- **Context Injection:** Historically (Phase 1 & 2), ops/bin/open was responsible for injecting specific agent personas (e.g., --agent=architect). However, DP-OPS-0022 removed this capability to "restore static behavior," pushing the dynamic logic down to the project layer.[3]

3.2. State Capture: ops/bin/dump

ops/bin/dump is the "Serializer" for the repository. It packs the file tree into a text format that agents can read.

- **Scopes:** It supports multiple scopes (platform, full, project) to optimize token usage.
- **Safety:** It serves as the primary "Receipt" mechanism. A Contractor cannot claim a task is done unless they provide a dump verifying the file state.[3]

3.3. Project Management: ops/bin/project

ops/bin/project is the "Work Management" interface. It is currently a wrapper around the logic in ops/lib/scripts/project.sh.

- **Current State (Broken):** Following the cleanup in DP-OPS-0022, ops/bin/project work attempts to pass --agent flags to ops/bin/open. Since ops/bin/open no longer accepts these flags, the project workflow is currently non-functional for specialized agents. **Fixing this is the primary objective of DP-OPS-0023**.[3]

IV. Forensic Analysis of the State of Play (Phase 2 to Phase 3)

A forensic review of SoP.md reveals the strategic arc of the last month, showing a deliberate move towards "Autopoiesis" (self-creation/maintenance).[3]

4.1. Phase 2: System Hardening

The entries from **DP-OPS-0010** to **DP-OPS-0020** demonstrate a focus on "Hardening" the core.

- **DP-OPS-0010:** Established the **Skills Subsystem** (SKILL.md), allowing the system to "learn" from mistakes by capturing reusable workflows.[3]
- **DP-OPS-0019:** Defined **Context Hazard** protocols. It recognized that loading the entire library of agents (docs/library/agents/) into every session was wasteful and confusing. It mandated that library files be excluded from the global context manifest (ops/lib/manifests/CONTEXT.md).[3]
- **DP-OPS-0020:** Implemented "Phase Two Doctrine," compressing the canon and enforcing strict linting styles.[3]

4.2. The Pivot: DP-OPS-0022

DP-OPS-0022 (executed Feb 05, 2026) was the "Factory Consolidation" event.

- **Logic Transplant:** It removed the agent-resolution logic from ops/bin/open and "transplanted" it into the library script ops/lib/scripts/project.sh.
- **Namespace Refactor:** The functions were renamed from resolve_agent_file to project_resolve_agent_file to strictly namespace them under the project domain.
- **The Gap:** While the logic was moved, it was not *activated*. The project binary was left pointing to the old (now removed) interface of open. This was a calculated risk ("Risk: Medium") documented in SoP.md to allow for atomic commits.[3]

V. Phase 3 Strategy: The Mechanistic Agent Model

The roadmap for Phase 3, detailed in DP-OPS-0023_NOTES.md, aims to solve the scaling problem of AI context.[3]

5.1. The "Project Election" Model

In Phase 2, the System decided which agents were available. In Phase 3, the **Project** decides.

- **Mechanism:** Every project will contain a STELA.md file (e.g., projects/P-001/STELA.md). This file acts as a "Machine Specification" or "Staffing Roster" for that specific project.[3]
- **Election:** If Project A needs an "Architect" and a "Security Reviewer," it lists them in its STELA.md. When ops/bin/project work is run, it reads this roster and dynamically injects *only* those two agents.

5.2. The Library Trinity

Phase 3 formalizes the operational knowledge base into three pillars, stored in docs/library/ [3]:

1. **Agents (docs/library/agents/):** The "Who." Personas, roles, and behavioral constraints (e.g., R-AGENT-06.md: Security Reviewer).[3]
2. **Tasks (docs/library/tasks/):** The "What." Recipes, checklists, and standard operating procedures (e.g., B-TASK-01.md: Code Review).[3]
3. **Skills (docs/library/SKILLS.md):** The "How." Specific command-line techniques, forensics, and drift prevention methods (e.g., S-LEARN-06.md: Git Forensics).[3]

5.3. Technical Implementation of Injection

Since ops/bin/open is now static (immutable), the ops/bin/project script must handle the injection "outside" of open.

- **Workflow:**
  1. project runs open to generate the base prompt file (OPEN-....txt).
  2. project resolves the required agents using the transplanted logic in

ops/lib/scripts/project.sh.
3. project reads the agent content.
4. project **appends** this content to the OPEN-....txt file generated in step 1.
5. project presents the final, concatenated prompt to the operator.

This approach maintains the purity of the System Gate (open) while allowing infinite flexibility at the Project Layer.[3]

VI. Analysis of DP-OPS-0023: The Bridge Task

DP-OPS-0023 is the execution order to operationalize the Phase 3 strategy. It is the "Activation" step for the logic transplanted in DP-OPS-0022.

6.1. Objectives

1. **Fix ops/bin/project:** Update cmd_work to stop using open --agent (which is dead) and start using the local project_resolve_agent_file logic.[3]
2. **Implement Injection:** Add the logic to append resolved agent content to the OPEN artifact.
3. **Refactor Linters:** Update tools/lint/context.sh and tools/lint/truth.sh to use robust git-root detection (git rev-parse --show-toplevel), fixing a fragility issue identified in DP-OPS-0022.[3]
4. **Skill Migration:** Finalize the move of SKILL.md to docs/library/SKILLS.md and update all pointers in llms.txt and ops/lib/scripts/skill.sh.[3]

6.2. Constraints and Allowlist

The Dispatch Packet must strictly adhere to the TASK.md template.

- **Allowlist:** The DP must explicitly list every file it touches. For this task, the allowlist includes ops/bin/project, ops/lib/scripts/project.sh, tools/lint/*, llms.txt, and docs/library/INDEX.md.[3]
- **Forbidden Zones:** No changes are allowed to projects/ (the payload) or .github/ (the CI workflows), as this is a platform-level operation.[3]

6.3. Verification

The DP must include a "Receipt" section requiring the worker to prove the changes.

- **Verification Command:** ./ops/bin/dump --scope=platform.
- **Linting:** bash tools/lint/truth.sh must pass with the new root-resolution logic.
- **Proof:** The worker must show that ops/bin/project work successfully generates an OPEN prompt containing agent context.[3]

VII. Draft Dispatch Packet (DP-OPS-0023)

The following Dispatch Packet is drafted in strict accordance with the TASK.md headers and the AGENTS.md behavioral standards. It serves as the executable contract for the Contractor.

DP-OPS-0023: Phase 3 Activation (Project Binary Alignment & Linter Refactor)

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/dp-ops-0023-phase3-activation

Base HEAD: dbce0b3d6

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - ops/bin/project

    - ops/bin/open
    - ops/lib/scripts/project.sh
    - tools/lint/context.sh
    - tools/lint/truth.sh
    - ops/lib/scripts/skill.sh

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN).

DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Activate Phase 3 "Project Election" model by refactoring ops/bin/project to inject agent context directly, and harden linters against path fragility.
- **Non-Goals (optional, drift-killer):**
    - No changes to docs/library/agents/ content.
    - No changes to projects/ payloads.
    - No functional changes to ops/bin/open (it remains static).

Target Files allowlist (hard gate)

- ops/bin/project
- ops/lib/scripts/project.sh
- tools/lint/context.sh
- tools/lint/truth.sh
- tools/lint/library.sh
- tools/verify.sh
- ops/lib/scripts/skill.sh
- llms.txt
- docs/library/INDEX.md
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
    - No edits to .github/
    - No edits to projects/ unless explicitly authorized
    - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**

    - Workers do not create/switch branches.
    - Workers do not stage/commit/merge/push.
    - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
    - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
    - STOP if required DP inputs are missing (do not guess).
    - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** DP-OPS-0022 removed dynamic agent logic from open, leaving project pointing to dead arguments (--agent). Logic was transplanted to project.sh but is unused. Linters fail if run outside root. SKILL.md location violates the "Library Trinity".
- **Drift:** ops/bin/project relies on open features that no longer exist. SKILL.md is in root.
- **Desired State:** ops/bin/project autonomously resolves and appends agent context to the OPEN prompt. Linters are path-agnostic. SKILL.md is consolidated in docs/library/.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. **Project Binary Alignment:**
    - Modify ops/bin/project function cmd_work.
    - Remove the --agent argument passed to ops/bin/open.
    - Implement logic to capture the output of ops/bin/open to the storage/handoff/ file.
    - Add logic to read the --agent argument from the project CLI.
    - Use project_resolve_agent_file (from ops/lib/scripts/project.sh) to find the agent file.
    - Use project_extract_agent_role (from ops/lib/scripts/project.sh) to read the content.
    - Append the agent content to the OPEN artifact with a clear header (e.g., ``).
    - Print the path of the final artifact.
2. **Linter Hardening:**
    - In tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh, tools/verify.sh:
    - Define REPO_ROOT="$(git rev-parse --show-toplevel)".
    - Force `cd "$REPO_ROOT" |

| exit 1` before any file operations.

- Ensure git ls-files calls are robust against unstaged deletions (update grep logic if needed).

3. **Skill Library Consolidation:**
    - Verify SKILL.md has been moved to docs/library/SKILLS.md (if not, move it).

    - Update ops/lib/scripts/skill.sh: Change SKILL_FILE variable to point to docs/library/SKILLS.md.
    - Update llms.txt: Replace SKILL.md reference with docs/library/SKILLS.md.
    - Update docs/library/INDEX.md: Update skills-template path.

Required Content (optional; use when doc/governance changes demand exact text/format):

- The injection in ops/bin/project must use the format:text

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
    - Updated ops/bin/project to natively resolve and inject agent context, decoupling it from open.
    - Refactored all linters to strictly resolve REPO_ROOT via git, preventing CWD errors.
    - Consolidated SKILL.md into docs/library/SKILLS.md and updated all system pointers.
    - Registered new paths in docs/library/INDEX.md and llms.txt.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform
- If dump refiners are used, include this fallback line in the DP:
    - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
    - Prefer: bash tools/lint/context.sh
    - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when TRUTH/canon surfaces touched; use repo-canonical command):
    - Prefer: bash tools/lint/truth.sh
    - If not found and required: **STOP** and ask (do not invent a substitute)

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted"

with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include SKILL.md (or docs/library/SKILLS.md).
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0023-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (required)
- storage/handoff/OPEN-[work-branch]-[hash].txt (required)
- storage/dumps/dump-full-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output

- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0023-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0023-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording;

copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

VIII. Strategic Implications

8.1. Scalability and Isolation
The execution of DP-OPS-0023 enables infinite scalability for the Stela platform. By decoupling agent injection from the system root (`open`), the repository avoids the "Monolithic Context" trap. Each project becomes a self-contained universe with its own staffing rules, yet they all share the same immutable `TRUTH.md` and operational tools.[2, 4]

8.2. Governance as Code
The shift to `STELA.md` specifications effectively implements "Governance as Code." Project staffing is no longer a matter of tribal knowledge ("Oh, you need the Security Agent for this repo") but a committed, version-controlled configuration. This reduces onboarding friction and ensures that governance rules are applied consistently across distributed teams.[6]

8.3. Future Trajectory
With Phase 3 active, the focus will shift to "Autopoietic Maintenance." Future tasks will involve creating `tools/lint/project.sh` to validate the `STELA.md` files, ensuring that projects do not reference dead agents or tasks. This "Meta-Governance" layer ensures that the system remains stable even as it grows in complexity.

IX. Conclusion

DP-OPS-0023 is not merely a refactoring task; it is the cornerstone of the Phase 3 architecture. By aligning the operational binaries with the "Library Trinity" and the "Project Election" model, it transforms Stela from a static repository into a dynamic, scalable "Software Factory." The rigorous adherence to the `TASK.md` template and the "Anti-Drift" protocols ensures that this transformation occurs without compromising the system's structural integrity or historical provenance.[1, 8]

Works cited

1. On the Precipice: Artificial Intelligence and the Climb to Modernize Nuclear Command, Control, and Communications - Federation of American Scientists, accessed February 5, 2026, https://fas.org/publication/on-the-precipice/
2. legacy-modernization · GitHub Topics, accessed February 5, 2026, https://github.com/topics/legacy-modernization

3. DP-OPS-0023_NOTES.md
4. (PDF) Agentic AI Under Pure Profit: No Governance, No Brakes, and the Unraveling of Accountability - ResearchGate, accessed February 5, 2026, https://www.researchgate.net/publication/399521146_Agentic_AI_Under_Pure_Profit_No_Governance_No_Brakes_and_the_Unraveling_of_Accountability
5. EU AI Act Compliance for Autonomous AI Systems: What C-Suite, accessed February 5, 2026, https://verityai.co/blog/eu-ai-act-compliance-autonomous-systems
6. The Equalizer and the Integrator: AI, Cybersecurity, and the Human Insight Imperative, accessed February 5, 2026, https://dev.to/narnaiezzsshaa/the-equalizer-and-the-integrator-ai-cybersecurity-and-the-human-insight-imperative-2o49
7. AI Governance: A framework for building responsible, ethical, fair, and transparent AI, accessed February 5, 2026, https://fusionlp.org/ai-governance-a-framework-for-building-responsible-ethical-fair-and-transparent-ai/
8. Legacy Architecture Modernisation With Strategic Domain-Driven Design | by Nick Tune, accessed February 5, 2026, https://medium.com/nick-tune-tech-strategy-blog/legacy-architecture-modernisation-with-strategic-domain-driven-design-3e7c05bb383f